

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – SCENARIO BASED ASSESSMENT

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO2 – Manipulation (BL1, BL2, BL3)	Assessment:	Assessment Tool 2 – Scenario Based Assessment
Weightage:	20% of CIA	Total Marks:	100
CO2 Statement:	Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull problems (BL1, BL2, BL3)		
Student Name:	Naveen Kumar S	Reg. No.:	192421030
Section / Batch:	B Tech IT	Date:	28/07/26

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given scenario and identify the computational problem involved.
- Apply appropriate Brute Force techniques for sorting, searching, Closest-Pair, and Convex-Hull problems wherever applicable.
- Clearly justify the chosen solution approach with proper reasoning and analytical interpretation.
- Demonstrate decision-making skills by comparing possible solutions and selecting the most suitable approach.
- Use diagrams, stepwise illustrations, or algorithmic representations wherever necessary.
- Maintain clarity, logical organization, and proper technical presentation throughout the answers.

Question Type	Focus Area	Questions
Scenario Based Assessment	Analyze real-world scenarios and apply Brute Force techniques for sorting, searching, Closest-Pair, and Convex-Hull problems	Q1

SECTION A – SCENARIO BASED ASSESSMENT

A document scanning tool matches keywords using brute-force string matching in large files.

- Explain how pattern matching is performed.
- Analyze time complexity for large inputs.
- Discuss inefficiency issues.

Code of Conduct

I Naveen Kumar S - 192421030 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Naveen Kumar S

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding & Context Analysis	25%	Thoroughly understands the scenario with accurate interpretation of all requirements	Clearly understands the scenario with minor gaps	Basic understanding ; some aspects of the scenario unclear	Poor understanding or misinterpretation of the scenario
Application of Concepts	30%	Applies appropriate concepts effectively to solve complex real world problems	Applies relevant concepts correctly with minor errors	Applies basic concepts but with limited accuracy	Fails to apply appropriate concepts
Analytical & Decision Making Skills	20%	Demonstrates strong analysis and selects optimal solutions with justification	Good analysis with reasonable solutions	Limited analysis; solutions lack justification	Poor or no analysis; incorrect decisions
Solution Design & Approach	15%	Well-structured, innovative, and feasible solution approach	Logical and structured solution with minor gaps	Basic solution with limited structure or feasibility	Unclear or incorrect solution approach
Clarity, Justification & Presentation	10%	Clear, well-justified explanation with strong reasoning	Clear explanation with some justification	Limited clarity and weak justification	Poorly explained with no justification

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding & Context Analysis	25%				
Application of Concepts	30%				
Analytical & Decision-Making Skills	20%				

Solution Design & Approach	15%				
Clarity, Justification & Presentation	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Brute-Force String Matching in a Document Scanning Tool

Introduction

Brute-force string matching is one of the simplest pattern matching techniques used in document scanning tools. It compares a keyword (pattern) with every possible position in a document (text) until the keyword is found or the document has been completely scanned.

a) Pattern Matching Process

1. Start comparing the keyword with the first character of the document.
2. Compare each character one by one.
3. If all characters match, the keyword is found.
4. If a mismatch occurs, shift the keyword by one position and repeat.
5. Continue until the pattern is found or the document ends.

Example:

Text: THIS IS A DOCUMENT SCANNING TOOL

Pattern: SCAN

The algorithm checks every possible starting position until 'SCAN' is matched.

b) Time Complexity Analysis

Let n be the length of the document and m be the length of the keyword.

Best Case: $O(m)$ – Pattern found at the first position.

Average Case: $O(n \times m)$ – Pattern found after several comparisons.

Worst Case: $O(n \times m)$ – Pattern not found or many partial matches occur.

For large documents, the number of character comparisons increases significantly, making the algorithm slower.

c) Inefficiency Issues

- Performs repeated character comparisons.
- Does not reuse information from previous mismatches.
- Slow for very large documents.
- Long keywords increase comparison time.
- Multiple keyword searches require scanning the document repeatedly.
- Higher CPU usage compared with advanced algorithms.

(a) Explain How Pattern Matching is Performed

Brute-force string matching compares the pattern (keyword) with the text character by character. The comparison begins from the first character of the document and continues until the pattern is either found or the entire document has been searched.

Pattern Matching Procedure

1. Read the document (text) and the keyword (pattern).
2. Start comparing the first character of the pattern with the first character of the document.
3. Continue comparing the remaining characters one by one.
4. If every character matches, the pattern is found successfully.
5. If a mismatch occurs at any position, stop comparing.
6. Shift the pattern one position to the right.
7. Repeat the comparison process from the new position.
8. Continue this process until:
 - the keyword is found, or
 - there are no more possible positions left in the document.

Example

Text

THIS IS A DOCUMENT SCANNING TOOL

Pattern

SCAN

Search Process

Attempt	Starting Position	Result
1	T	Mismatch
2	H	Mismatch
3	I	Mismatch
...	...	...
Final Attempt	S	Complete Match Found

The algorithm checks every possible position until it successfully matches the keyword **SCAN**.

Characteristics of Brute-Force Pattern Matching

- Very easy to implement.
- Does not require preprocessing.
- Works on both sorted and unsorted text.
- Suitable for small documents.
- Searches characters sequentially.
- Stops immediately after finding the required keyword (if only one occurrence is needed).
- Requires no extra memory except for a few variables.

(b) Analyze Time Complexity for Large Inputs

Assume:

- **n** = Length of the document
- **m** = Length of the keyword

The performance depends on where the keyword appears.

Best Case

Time Complexity

$O(m)$ (often simplified to **$O(1)$** when m is treated as a fixed keyword length)

Explanation

The keyword appears at the beginning of the document.

Example:

Text : COMPUTER SCIENCE

Pattern : COMPUTER

Only one comparison sequence is required.

Average Case

Time Complexity

$O(n \times m)$

Explanation

The keyword appears somewhere in the middle of the document.

The algorithm compares several characters before finding the correct position.

Worst Case

Time Complexity

$O(n \times m)$

Explanation

The keyword is not present or repeated partial matches occur.

Example

Text : AAAAAAAAAAAAAAAAAA

Pattern : AAAAB

Almost every position requires multiple comparisons before failing.

Time Complexity Table

Case	Complexity	Description
Best Case	$O(m)$	Pattern found immediately
Average Case	$O(n \times m)$	Pattern found after several attempts
Worst Case	$O(n \times m)$	Pattern absent or found at the end

Space Complexity

The brute-force algorithm uses only a few variables during execution.

Space Complexity = $O(1)$

This makes it memory-efficient despite its slower execution time.

(c) Discuss Inefficiency Issues

Although brute-force string matching is simple, it has several disadvantages when processing large documents.

1. Repeated Comparisons

The algorithm repeatedly compares the same characters after every mismatch.

This increases the execution time significantly.

2. Slow Performance

Large documents may contain millions of characters.

The algorithm checks nearly every position, making the search slow.

3. No Preprocessing

Unlike advanced algorithms, brute-force matching does not analyse the pattern before searching.

Therefore, it cannot skip unnecessary comparisons.

4. Poor Scalability

As document size increases, the execution time increases almost proportionally.

This makes it unsuitable for search engines or large document repositories.

5. Inefficient for Multiple Searches

If several keywords need to be searched, the document is scanned repeatedly.

This causes unnecessary processing.

6. Higher CPU Utilization

Continuous character-by-character comparisons increase CPU usage.

This reduces overall system performance.

7. Redundant Character Checking

Previously compared characters may be compared again after shifting the pattern.

This redundancy wastes processing time.

8. Poor Performance with Long Patterns

Longer keywords require more comparisons.

For example, searching a 20-character keyword is much slower than searching a 4-character keyword.

9. Unsuitable for Real-Time Applications

Applications such as search engines, antivirus software, plagiarism detection, and document indexing require faster search algorithms.

Brute-force matching cannot meet these performance requirements efficiently.

10. Better Alternatives

More efficient string matching algorithms include:

- **Knuth–Morris–Pratt (KMP)** – avoids unnecessary comparisons using preprocessing.
 - **Boyer–Moore** – skips multiple characters after a mismatch.
 - **Rabin–Karp** – uses hashing for efficient searching.
 - **Aho–Corasick** – efficiently searches for multiple keywords simultaneously.
-

Advantages

- Simple to understand and implement.
 - Requires no preprocessing.
 - Uses constant extra memory ($O(1)$).
 - Effective for small documents.
 - Suitable when searches are infrequent.
 - Works regardless of whether the text is sorted.
-

Disadvantages

- High execution time for large documents.
- Repeated character comparisons.
- Poor scalability.
- Inefficient for long keywords.
- Not suitable for multiple keyword searches.
- Slower than advanced pattern-matching algorithms.

Conclusion

Brute-force string matching is simple and easy to implement but becomes inefficient for large datasets because of its $O(n \times m)$ average and worst-case complexity. Modern algorithms such as Knuth–Morris–Pratt (KMP), Boyer–Moore, and Rabin–Karp improve performance by reducing unnecessary comparisons.